

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.20

CONNECTION

 Copy as Markdown

Connection

The connection details and password for the database live in the `config` directory of your server package, and Serverpod connects using them when the server starts. For local development, no setup is needed by default: the server runs an [embedded PostgreSQL](#) and manages it for you. This page is for connecting to a database you run yourself: a local or Docker Postgres, a remote server, or a managed instance such as Cloud SQL or RDS.

Connection details

Each environment configuration contains a `database` keyword that specifies the connection details. For your development build you can find the connection details in the `config/development.yaml` file.

```
...
database:
  host: localhost
  port: 8090
  name: <YOUR_PROJECT_NAME>
  user: postgres
...
```

The `name` refers to the database name, `host` is the domain name or IP address pointing to your Postgres instance, `port` is the port that Postgres is listening to, and `user` is the username that is used to connect to the database.

CAUTION

By default, Postgres listens on port 5432. Serverpod's development setups use port 8090 to avoid conflicts. If you host your own instance, double-check that the correct port is specified in your configuration files.



Ask AI

SQLite projects need no connection details beyond a file path, and no password; see [Database backends](#) for choosing and configuring a backend.

Configure search paths

If using Postgres, you can customize the search paths for your database connection, which helps when you work with multiple schemas. By default, Postgres uses the `public` schema unless otherwise specified.

To override this, use the optional `searchPaths` setting in your configuration:

```
...
database:
  host: localhost
  port: 8090
  name: <YOUR_PROJECT_NAME>
  user: postgres
  searchPaths: custom, public
...
```

In this example, Postgres looks for tables in the `custom` schema first, and falls back to `public` if needed.

TIP

It is also possible to set the search paths using [runtime parameters](#) directly on the server startup (or on a specific transaction). If the paths are set on both the configuration file and as runtime parameters, the runtime parameters take precedence.

Configure connection pool size

By default, Serverpod uses a connection pool with a maximum of 10 connections to the database. You can customize this limit using the `maxConnectionCount` setting:

```
...
database:
  host: localhost
  port: 8090
  name: <YOUR_PROJECT_NAME>
  user: postgres
  maxConnectionCount: 20
...
```

To allow unlimited connections, set `maxConnectionCount` to `0` or a negative value. You can also configure this setting via the environment variable `SERVERPOD_DATABASE_MAX_CONNECTION_COUNT`.

On SQLite, this setting controls the number of read-only transactions that can run concurrently. Only one write transaction can run at a time.

Database password

The database password is stored in a separate file called `passwords.yaml` in the same `config` directory. The password for each environment is stored under the `database` keyword in the file.

```
...
development:
  database: '<MY DATABASE PASSWORD>'
...
```

The development database

By default, the development database is an [embedded PostgreSQL](#), started and stopped with the server; see [Running your server](#). Nothing on this page is needed for it.

Projects set up with the Docker alternative instead run their development database from the server package's `docker-compose.yaml`. To use it in a project configured for the embedded PostgreSQL, first remove `dataPath` from the run mode's configuration. Start the database from the root of the `server` package:

```
$ docker compose up --build --detach
```

To stop the database run:

```
$ docker compose stop
```

To remove the database and **delete** all associated data, run:

```
$ docker compose down -v
```

Connect to a custom Postgres instance

You can connect to any Postgres instance you run or rent. There are a few things to take into consideration:

- Make sure that your Postgres instance is up and running and is reachable from your Serverpod server.
- You will need to create a user with a password, and a database.

Connect to a local Postgres server

If you want to connect to a local Postgres server (with the default setup), the `development.yaml` works fine if you set the correct port, user, and database, and update the password in the `passwords.yaml` file.

Connect to a remote Postgres server

To connect to a remote Postgres server (that you have installed on a VPS or VDS), you need to follow a couple of steps:

- Make sure that the Postgres server has a reachable network address and that it accepts incoming traffic.
- You may need to open the database port on the machine. This may include configuring its firewall.
- Update your Serverpod `database` config to use the public network address, database name, port, user, and password.

Connect to Google Cloud SQL

You can connect to a Google Cloud SQL Postgres instance in two ways:

1. Setting up the *Public IP Authorized networks* (with your Serverpod server IP) and changing the database host string to the *Cloud SQL public IP*.
2. Using the *Connection String* if you are hosting your Serverpod server on Google Cloud Run and changing the database host string to the Cloud SQL: `/cloudsql/my-project:server-location:database-name/.s.PGSQL.5432`.

The next step is to update the database password in `passwords.yaml` and the connection details for the desired environment in the `config` folder.

ⓘ INFO

If you are using the `isUnixSocket` don't forget to add `"/s.PGSQL.5432"` to the end of the `host` IP address. Otherwise, your Google Cloud Run instance will not be able to connect to the database.

Connect to AWS RDS

You can connect to an AWS RDS Instance in two ways:

1. Enable public access to the database and configure VPC/Subnets to accept your Serverpod's IP address.
2. Use the Endpoint `database-name.some-unique-id.server-location.rds.amazonaws.com` to connect to it from AWS ECS.

The next step is to update the database password in `passwords.yaml` and the connection details for the desired environment in the `config` folder.